Technical Report on Remote Sensing Image Segmentation Experiments

1.1 Problem Statement

Remote sensing image segmentation is crucial for various applications, such as land cover classification, environmental monitoring, and urban planning. Traditional segmentation models require dense pixel-wise annotations, which are expensive and time-consuming to obtain. To address this, we explore a sparse annotation approach where only a subset of pixels is labeled, leveraging a custom Partial Cross-Entropy Loss to improve segmentation performance. Thus, in this study, we implement Partial Cross-Entropy Loss and integrate it into a remote sensing segmentation network. The goal is to investigate how this loss function influences model performance compared to standard approaches. Additionally, we explore how hyperparameters such as batch size and learning rate impact segmentation performance.

1.2 Objective

The primary goals of this study are:

1. To implement U-Net with ResNet-34 encoder for remote sensing image segmentation.
2. To incorporate Partial Cross-Entropy Loss to handle sparse annotations effectively.
3. To conduct experiments by varying key hyperparameters such as learning rate and batch size.
4. To analyze the impact of sparse labeling on model performance and evaluate its generalizability.

Methodology

2.1 Dataset

We use the GID-15 dataset, a widely used remote sensing dataset containing high-resolution images with 15 land cover classes, including urban areas, vegetation, water bodies, and farmland.

Since the dataset consists of large images, the following preprocessing steps were applied:

1. Resizing: All images were resized to 512×512 pixels to ensure consistency.
2. Data Augmentation: Applied horizontal flipping to enhance generalization.
3. Normalization: Pixel values were scaled to [0,1] range for stable training.

Since fully labeled segmentation masks are costly to obtain, we simulate a sparse annotation scenario by randomly selecting 500 labeled pixels per image. These labeled pixels are used for training, while the rest are ignored using Partial Cross-Entropy Loss.

2.2 Model Architecture

U-Net with ResNet-34 Encoder

We utilize Segmentation Models PyTorch (SMP) to implement a U-Net model with a ResNet-34 encoder pre-trained on ImageNet.

1. ResNet-34 extracts high-level spatial features from images.
2. A series of upsampling and convolution layers reconstruct the segmentation map.
3. Final Layer: A 1×1 convolution to output a segmentation mask of 15 classes

Partial Cross-Entropy Loss

Cross-entropy loss is commonly used in segmentation tasks but assumes fully labeled data. However, in remote sensing applications, labels may be sparse or partially annotated. Hence, in our sparse annotation setup, most pixels remain unlabeled. Partial Cross-Entropy Loss (PCE) modifies the standard cross-entropy by computing loss only for known labels while ignoring unknown, uncertain pixels or in our case pixels that serve as just the background of the image (index 0).

This ensures that the model does not learn from uncertain or missing labels, leading to better generalization.

Implementation:

```

class PartialCrossEntropyLoss(nn.Module):

    def __init__(self, ignore_index=0):  # Ignore unlabeled pixels (background)

        super(PartialCrossEntropyLoss, self).__init__()

        self.ignore_index = ignore_index

        self.criterion = nn.CrossEntropyLoss(reduction='none')


    def forward(self, predictions, target):

        loss = self.criterion(predictions, target)
```

```
    valid_mask = (target != self.ignore_index).float()  # Only labeled pixels
    return torch.sum(loss * valid_mask) / (torch.sum(valid_mask) + 1e-6)
```

2.4 Training Strategy

Hyperparameter Tuning

We perform hyperparameter tuning to optimize model performance. The following hyperparameters are varied:

- Learning Rate: {0.001, 0.0005}
- Batch Size: {8, 16}
- Epochs: 10

All hyperparameter settings and results were logged using MLflow for experiment tracking.

```mlflow.log_param("learning_rate", lr)
mlflow.log_param("batch_size", bs)
mlflow.log_param("num_epochs", num_epochs)
mlflow.log_param("patch_size", 512)
```

3.0 Experiment Setup and Results

We did not make use of the Test loss but validation loss. Since the GID-15 test set lacks ground truth labels, validation loss served as the main metric to compare hyperparameter configurations.

Experiment 1: Effect of Batch Size on Performance

Hypothesis

Increasing batch size will improve convergence speed but may generalize poorly due to fewer updates.

Findings

1. Batch Size 8 consistently achieved lower validation loss in the best run (approximately 0.967).
2. Batch Size 16 had higher validation loss in the runs shown.

In [93]:

```
# print best run information
display(best_run_df[["run_id", "status", "metrics.val_loss", "params.learning_rate", "tags.
mlflow.runName"]])
```

	run_id	status	metrics.val_loss	params.learning_rate	tags.mlflow.runName
0	c54b2a082add44019d2238e00e37bed3	FINISHED	0.967428	0.0005	lr_0.0005_bs_8

Conclusion: A smaller batch size (8) led to better validation performance in these experiments, aligning with the idea that smaller batches can improve generalization by introducing noisier gradient updates.

Experiment 2: Effect of Learning Rate on Convergence

Hypothesis

- Lower learning rates provide stable training but slower convergence.
- Higher learning rates speed up training but cause instability.

Findings

1. Lower LR (0.0005) gave the best overall validation loss (around 0.97).
2. Higher LR (0.001) produced higher validation losses, indicating less stable or optimal training.

	run_id	status	metrics.val_loss	params.learning_rate	tags.mlflow.runName
0	4492a933451147af9fdf064ee8db0052	FINISHED	2.744635	0.0005	lr_0.0005_bs_16
1	c54b2a082add44019d2238e00e37bed3	FINISHED	0.967428	0.0005	lr_0.0005_bs_8
2	8cb47dd4ed084177ae2a772e8a12a6b8	FINISHED	3.191504	0.001	lr_0.001_bs_16
3	7d28abc61bef452fa5f4a8f6c303eb95	FINISHED	2.746076	0.001	lr_0.001_bs_8
4	f77cd2a1bd4f41b2ae17f92b0a9b1dfa	FAILED	NaN	0.001	lr_0.001_bs_8
5	2a14ff6ce50a4128a5f4b08a5aac3fdf	FAILED	NaN	0.001	lr_0.001_bs_8

Conclusion: 0.0005 was more stable and yielded the lowest validation loss. Although 0.001 might converge faster, it showed greater fluctuations and higher final loss.

Model Training and Validation Loss Chart

The following chart shows that the training loss (blue line) generally remains between 2.2 and 2.7, indicating that the model is consistently fitting the training data with some variation across epochs.

- Training Loss (Blue Line): Generally sits around 2.2–2.7, indicating the model fits the training data but with some variation per epoch.
- Validation Loss (Orange Line): Fluctuates significantly, reflecting how sensitive the model is to different batches or hyperparameter choices.
- The lowest valley in the orange line correlates with the best run (lr=0.0005, bs=8).
- No Overfitting is evident since the validation curve doesn't consistently rise above training loss by a large margin, but there is noticeable epoch-to-epoch volatility.

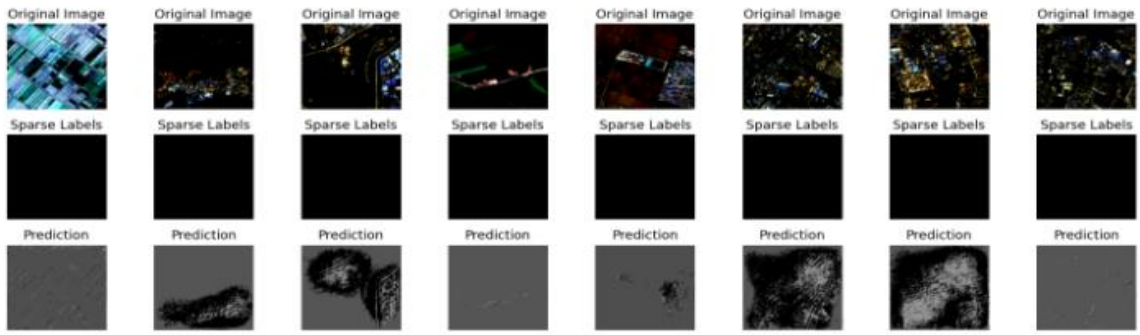
3.3 Visualizing Predictions

To evaluate model performance, we compare predicted segmentation maps against ground truth sparse labels.

Prediction Output

- Row 1: Original Image
- Row 2: Sparse Labeled Mask
- Row 3: Model Prediction

```
[100]: val_loader    = DataLoader(val_dataset, batch_size=bs, shuffle=False, generator=generator)
# visualize predictions using the best model
visualize_predictions(best_model, val_loader)
```



4.1 Key Insights

1. Best Run: Learning rate = 0.0005, batch size = 8, with a validation loss of 0.967.

2. Higher LR (0.001) caused the model to converge to higher validation loss.

2. Larger batch size (16) also correlated with higher validation loss in these trials.

3. Sparse Annotations with Partial Cross-Entropy Loss were still sufficient to achieve meaningful segmentation performance, but stable hyperparameters (lower LR, smaller batch) proved crucial.

In summary**,** lower learning rate (0.0005) and smaller batch size (8) provided the most stable and lowest validation loss in this set of experiments, confirming the hypotheses that lower LR ensures more reliable convergence and smaller batch sizes can improve generalization.

4.2 Limitations

- Current experiments were conducted on GID-15, and generalizability needs further validation on other datasets.
- Future research should test different architectures (e.g., DeepLabV3) for improved segmentation accuracy.

References

- GID-15 Dataset: GID-15 Dataset
- U-Net segmentation model: U-Net
- Pytorch: Pytorch